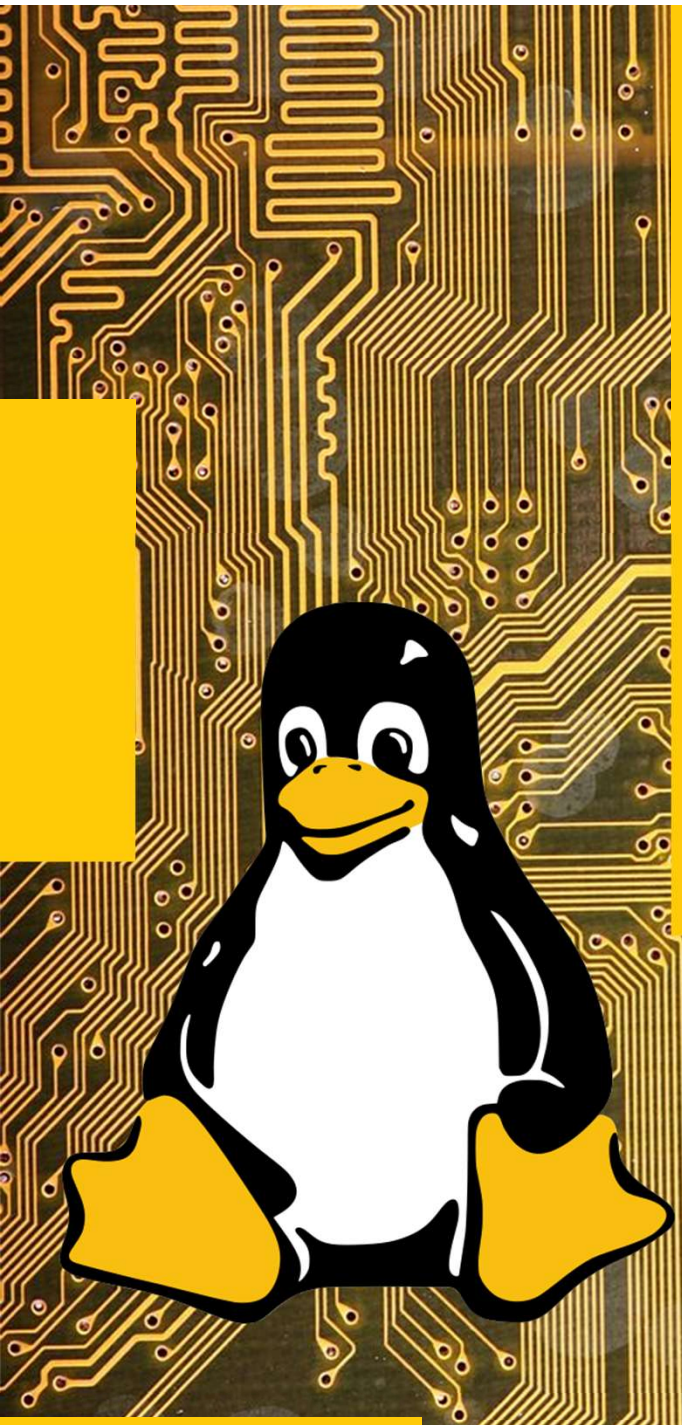


INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

The Shell and CPU Scheduling (TLPI Ch 34; OSTEP Ch 7, 8)

INSTRUCTOR: Jalolliddin Yusupov

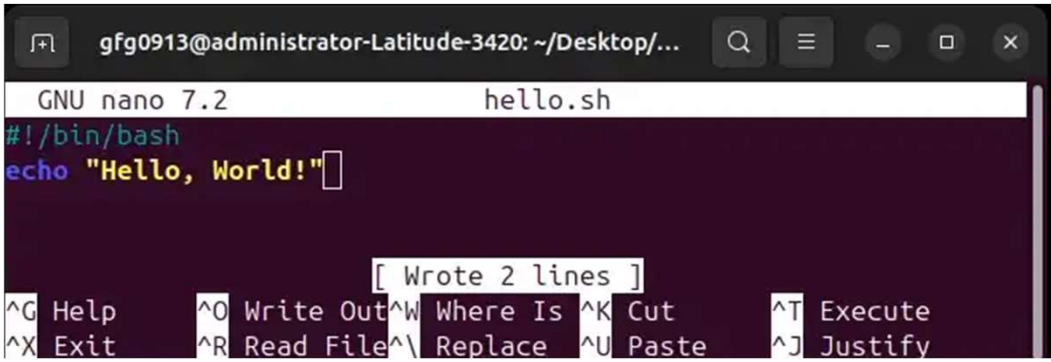
A Shell Is a Loop Around Four System Calls

A shell is a user-space program. Everything it does, you can do in C!

You can make a process, replace its program, wait for it, and bury it…..

Today we look at the program that does all of that for a living, thousands of times a day, on your machine. **The shell.**

- It allows communicating with the O.S.
  - interactively
  - by means of a script



THE REPL (READ–EVAL–PRINT LOOP), IN PSEUDO-CODE

```
while (1) {
    line = read_line("$ ");           // 1. read
    argv = parse(line);                // 2. split into words (S7)
    if (is_builtin(argv)) { run_here(argv); continue; }
    pid = fork();                      // 3. a copy of the shell
    if (pid == 0)
        execvp(argv[0], argv);         //    child: new brain
    waitpid(pid, &status, WUNTRACED); // 4. parent: next of kin
    last_status = decode(status);      //    becomes $?
}
```

`$?` — the shell variable holding the **last command's exit status** (last week's death certificate).

Everything a shell does is fork, exec, wait — and a variable named $?.

$?: The Canonical Table, Exit Codes

```
true          → $? = 0
false         → $? = 1
./nope        → $? = 127   (127: not found)
./x (no +x)   → $? = 126   (126: not executable)
kill -9 sleep → $? = 137   (128+9)
kill -TERM    → $? = 143   (128+15)
SIGSEGV       → $? = 139   (128+11)
```

THE BLOOD TYPE OF A DEAD COMMAND

`0`	**Success.** The only value `if` treats as true.
`1-125`	The program's own meaning (`grep` : 1 = no match, 2 = error).
`126`	Found, but **not executable** (no `x` bit, or a directory).
`127`	**Not found** — typo, or not on `PATH` .
`128+N`	**Killed by signal N**: 130 Ctrl-C (2), 137 `kill -9` , 139 segfault (11), 143 `SIGTERM` (15).

`$?` — the shell variable holding the **last command's exit status** (last week's death certificate).

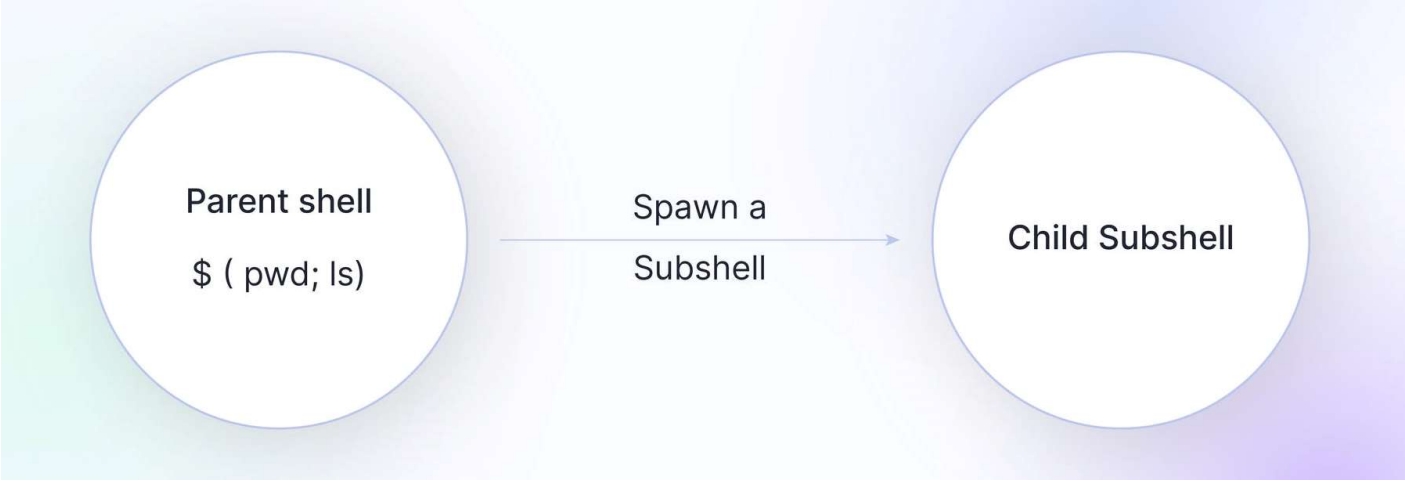
Everything a shell does is fork, exec, wait — and a variable named $?.

What is a subshell?

A **subshell** is a child copy of the current shell process. It inherits everything (variables, functions, options) but any changes it makes are **invisible** to the parent.

```
# Parentheses create a subshell
( x=42; echo "inside: $x" )
echo "outside: $x"    # x is unset here

# Braces group in the CURRENT shell
{ x=42; echo "inside: $x"; }
echo "outside: $x"    # x is 42 here
```



Sessions & Process Groups

How the kernel organizes processes around terminals

PROCESS

A running program. Has a **PID**.

PROCESS GROUP

A set of processes with a shared **PGID**. Signals can target the whole group. One member is the **group leader** (PGID == its PID).

SESSION

A collection of process groups, with a shared **SID**. Usually tied to one controlling terminal. Leader is the **session leader**.

```
$ ps -eo pid,ppid,pgid,sid,tty,stat,comm
  PID  PPID  PGID   SID TT       STAT COMMAND
 1234  1200  1234  1234 pts/0    Ss   bash      ← session & group leader  S
 1500  1234  1500  1234 pts/0    S+   vim       ← own group, foreground  S+
 1600  1234  1600  1234 pts/0    S    sleep     ← own group, background
```

`-e` : Selects all processes.

`-o` : Specifies a custom output format displaying the exact columns requested.

Signals to a group: `kill -INT -1500` sends SIGINT to the whole group (note the minus sign).

Job Control: Foreground, background, suspended

Juggling multiple processes on one terminal

FOREGROUND

Owns the terminal. Reads input, receives `Ctrl` + `C`.

Started normally: `vim file`

BACKGROUND

Runs but does not own the terminal. Trying to read → `SIGTTIN` (stops it).

Started with `&`: `sleep 60 &`

STOPPED

Suspended by `Ctrl` + `Z` (SIGTSTP). Frozen in memory.

Resume with `fg` or `bg`.

```
$ long-task
^Z                          # suspend
[1]+  Stopped   long-task

$ jobs                      # list jobs
[1]-  Stopped   long-task
[2]+  Running   sleep 60 &

$ bg %1                     # resume #1 in background
$ fg %2                     # bring #2 to foreground
$ fg                        # most recent job (the + one)
```

Daemons: Long-running background services

- Has **no controlling terminal**
- Runs in the background, usually started at boot
- Is an orphan — its parent is `init` (PID 1)
- Examples: `sshd`, `crond`, `nginx`, `systemd`



Daemonizing from the shell

1.Open a terminal and create a new file:

```
sudo nano /etc/systemd/system/telegram-bot.service
```

2. Paste the following configuration

```
1 [Unit]
2 Description=Telegram Bot
3
4 [Service]
5 ExecStart=/usr/local/bin/telegram-bot.sh
```

3. Enable it so it starts automatically when the server reboots:

```
sudo systemctl enable telegram-bot
```

Systemd is a system and service manager for Linux that initializes the operating system and manages background services after the kernel boots.

The `trap` builtin

Register a command to run when the shell receives a signal

```
bash

  1  trap 'WHAT TO DO'  WHEN TO DO IT
```

- **WHAT TO DO:** The cleanup commands (e.g., delete a file, kill a background process).
- **WHEN TO DO IT:** The "Signal" (e.g., `SIGINT` for Ctrl+C, or `EXIT` for when the script finishes).

```
positive#   trap 'echo " 🛑 Caught Ctrl+C!"; ' SIGINT
 🛑 Caught Ctrl+C!
```

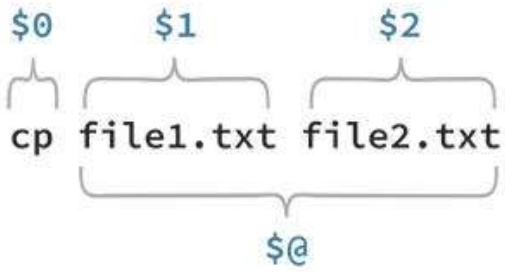
What is a script? The first bash program

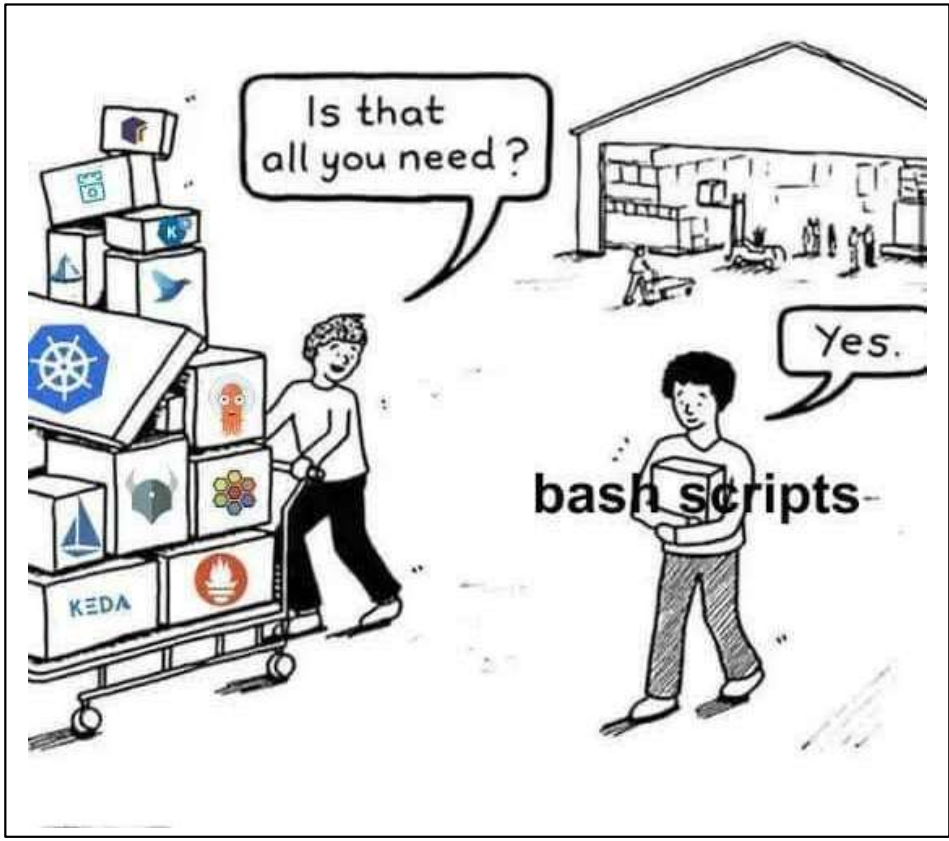
- A script is a text file of commands that the shell reads and executes.
- The **shebang** tells the kernel which interpreter to use.
- You can run it as `./script.sh` after `chmod +x`, or as `bash script.sh`.

```
#!/usr/bin/env bash
echo "hello from $0"
echo "first arg: $1"
echo "all args: $@"
```

hello.sh

```
$ chmod 700 hello.sh
$ ./hello.sh one two
hello from ./hello.sh
first arg: one
all args: one two
```

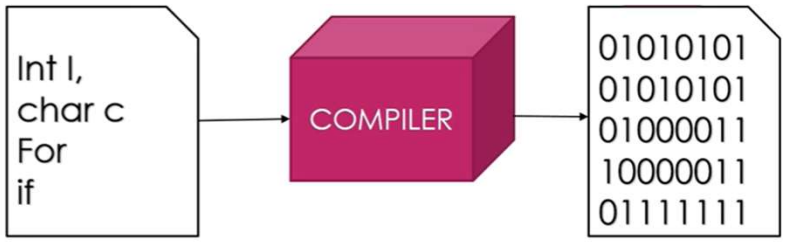


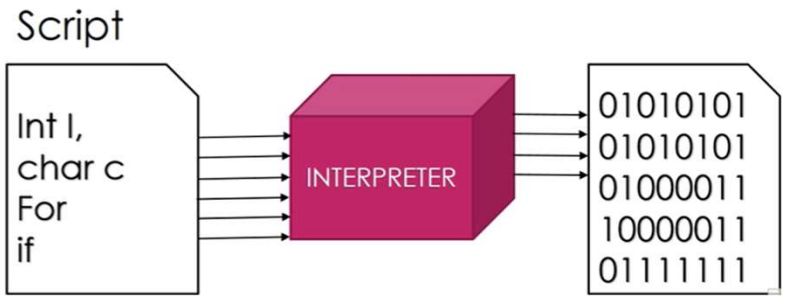


The shebang only matters when the file is executed directly. `bash hello.sh` ignores it.

Programming or Scripting ?

- bash is not only an excellent command line shell, but a scripting language in itself. Shell scripting allows us to use the shell's abilities and to automate a lot of tasks that would otherwise require a lot of commands.
- Difference between programming and scripting languages:





- **Programming languages** are generally a lot **more powerful and a lot faster than scripting languages**. Programming languages generally start from source code and are compiled into an executable. This executable is not easily ported into different operating systems.
- A **scripting language** also starts from source code, but is not compiled into an executable. Rather, an interpreter reads the instructions in the source file and executes each instruction. Interpreted programs are generally slower than compiled programs. The main advantage is that you can easily port the source file to any operating system. bash is a scripting language. Other examples of scripting languages are *Perl, Lisp, and Tcl.*

The second bash program

- We write a program that copies all files into a directory, and then deletes the directory along with its contents. This can be done with the following commands:

```
$ mkdir trash
$ cp * trash
$ rm -rf trash

$ mkdir trash
```

- Instead of having to type all that interactively on the shell, write a shell program instead:

```
$ cat trash.sh

#!/bin/bash
cp * trash
rm -rf trash
mkdir trash

echo "Deleted all files!"
```

Variables and quoting — the minimum

```
name="Ana"
count=3
echo "$name has $count items"
echo '$name is not expanded here'
```

- No spaces around = in assignments.
- "$var" expands and keeps spaces together.
- '$var' is literal — no expansion.
- Always prefer "$@" for all arguments, and "$var" for variables.

The export command

- The export command puts a variable into the environment so it will be accessible to child processes. For instance:

```
$ x=hello
$ bash              # Run a child shell.
$ echo $x           # Nothing in x.
$ exit              # Return to parent.
$ export x
$ bash
$ echo $x
hello               # It's there
```

- If the child modifies x, it will not modify the parent's original value. Verify this by changing x in the following way:

```
$ x=ciao
$ exit
$ echo $x
hello
```

Environmental Variables

```
$ echo $SHELL
/bin/bash
$ echo $PATH
/usr/X11R6/bin:/usr/local/bin:/bin:/usr/bi
```

- There are two types of variables:
- Local variables
- Environmental variables
- Environmental variables are set by the system and can usually be found by using the env command. Environmental variables hold special values. For instance:

Environmental Variables

- HOME: The default argument (home directory) for cd.
- PATH: The search path for commands. It is a colon-separated list of directories that are searched when you type a command.

- Usually, we type in the commands in the following way:

```
$ ./command
```

- By setting PATH=$PATH:. our working directory is included in the search path for commands, and we simply type:

```
$ command
```

- If we type in

```
$ mkdir ~/bin
```

Read command

- The read command allows you to prompt for input and store it in a variable.

- Example:

```
#!/bin/bash
echo -n "Enter name of file to delete: "
read file
echo "Type 'y' to remove it, 'n' to change your mind ... "
rm -i $file
echo "That was YOUR decision!"
```

- Line 2 prompts for a string that is read in line 3. Line 4 uses the interactive remove (rm -i) to ask the user for confirmation.

Arithmetic Evaluation

- The let statement can be used to do mathematical functions:

```
$ let X=10+2*7
$ echo $X
24
$ let Y=X+2*4
$ echo $Y
32
```

- An arithmetic expression can be evaluated by $[expression] or $((expression))

```
$ echo "$((123+20))"
143
$ VALORE=$[123+20]
$ echo "$[123*$VALORE]"
17589
```

Conditional Statements

- Conditionals let us decide whether to perform an action or not, this decision is taken by evaluating an expression. The most basic form is:

```
if [ expression ];
then
        statements
elif [ expression ];
then
        statements
else
        statements
fi
```

- the elif (else if) and else sections are optional
- Put spaces after [and before], and around the operators and operands.

Expressions

- An expression can be: String comparison, Numeric comparison, File operators and Logical operators and it is represented by [expression]:

- String Comparisons:

= compare if two strings are equal
!= compare if two strings are not equal
-n evaluate if string length is greater than zero
-z evaluate if string length is equal to zero

- Examples:

[s1 = s2] (true if s1 same as s2, else false)
[s1 != s2] (true if s1 not same as s2, else false)
[s1] (true if s1 is not empty, else false)
[-n s1] (true if s1 has a length greater then 0, else false)
[-z s2] (true if s2 has a length of 0, otherwise false)

Expressions

- Number Comparisons:

-eq	compare if two numbers are equal
-ge	compare if one number is greater than or equal to a number
-le	compare if one number is less than or equal to a number
-ne	compare if two numbers are not equal
-gt	compare if one number is greater than another number
-lt	compare if one number is less than another number

- Examples:

[n1 -eq n2]	(true if n1 same as n2, else false)
[n1 -ge n2]	(true if n1greater then or equal to n2, else false)
[n1 -le n2]	(true if n1 less then or equal to n2, else false)
[n1 -ne n2]	(true if n1 is not same as n2, else false)
[n1 -gt n2]	(true if n1 greater then n2, else false)
[n1 -lt n2]	(true if n1 less then n2, else false)

Homework:

```
$ cat user.sh
 #!/bin/bash
    echo -n "Enter your login name: "
    read name
    if [ "$name" = "$USER" ];
    then
            echo "Hello, $name. How are you today ?"
    else
            echo "You are not $USER, so who are you ?"
    fi
$ cat number.sh
#!/bin/bash
    echo -n "Enter a number 1 < x < 10: "
    read num
    if [ "$num" -lt 10 ]; then
            if [ "$num" -gt 1 ]; then
                    echo "$num*$num=$(($num*$num))"
            else
                    echo "Wrong insertion !"
            fi
    else
            echo "Wrong insertion !"
    fi
```

Expressions

- Files operators:

-d check if path given is a directory
-f check if path given is a file
-e check if file name exists
-r check if read permission is set for file or directory
-s check if a file has a length greater than 0
-w check if write permission is set for a file or directory
-x check if execute permission is set for a file or directory

- Examples:

[-d fname] (true if fname is a directory, otherwise false)
[-f fname] (true if fname is a file, otherwise false)
[-e fname] (true if fname exists, otherwise false)
[-s fname] (true if fname length is greater then 0, else false)
[-r fname] (true if fname has the read permission, else false)
[-w fname] (true if fname has the write permission, else false)
[-x fname] (true if fname has the execute permission, else false)

Expressions

- Logical operators:

```
!       negate (NOT) a logical expression
-a logically AND two logical expressions
-o      logically OR two logical expressions
```

Example:

```
#!/bin/bash
   echo -n "Enter a number 1 < x < 10:"
   read num
   if [ "$num" -gt 1 –a "$num" -lt 10 ];
   then
          echo "$num*$num=$(($num*$num))"
   else
          echo "Wrong insertion !"
   fi
```

```
# Run B only if A succeeded
A && B

# Run B only if A failed
A || B

# Classic: try, or fall back
cd /srv/app || exit 1
```

Iteration Statements

- The for structure is used when you are looping through a range of variables.

```
for var in list
   do
     statements
   done
```

- statements are executed with var set to each value in the list.
- Example

```
#!/bin/bash
   let sum=0
   for num in 1 2 3 4 5
      do
        let "sum = $sum + $num"
      done
   echo $sum
```

Iteration Statements

```
#!/bin/bash
   for x in paper pencil pen
  do
     echo "The value of variable x is: $x"
     sleep 1
   done
```

- if the list part is left off, var is set to each parameter passed to the script ($1, $2, $3,...)

```
$ cat for1.sh
   #!/bin/bash
   for x
   do
     echo "The value of variable x is: $x"
     sleep 1
   done
$ for1.sh arg1 arg2
   The value of variable x is: arg1
   The value of variable x is: arg2
```

Using Arrays with Loops

- In the bash shell, we may use arrays. The simplest way to create one is using one of the two subscripts:

```
pet[0]=dog
pet[1]=cat
pet[2]=fish
pet=(dog cat fish)
```

- We may have up to 1024 elements. To extract a value, type ${arrayname[i]}

```
$ echo ${pet[0]}
    dog
```

- To extract all the elements, use an asterisk as:

```
echo ${arrayname[*]}
```

- We can combine arrays with loops using a for loop:

```
for x in ${arrayname[*]}
    do
            ...
    done
```

Functions

- Functions make scripts easier to maintain. Basically it breaks up the program into smaller pieces. A function performs an action defined by you, and it can return a value if you wish.

```
#!/bin/bash
  hello()
  {
  echo "You are in function hello()"
  }

  echo "Calling function hello()..."
  hello
  echo "You are now out of function hello()"
```

- In the above, we called the hello() function by name by using the line: hello . When this line is executed, bash searches the script for the line hello(). It finds it right at the top, and executes its contents.



Questions?